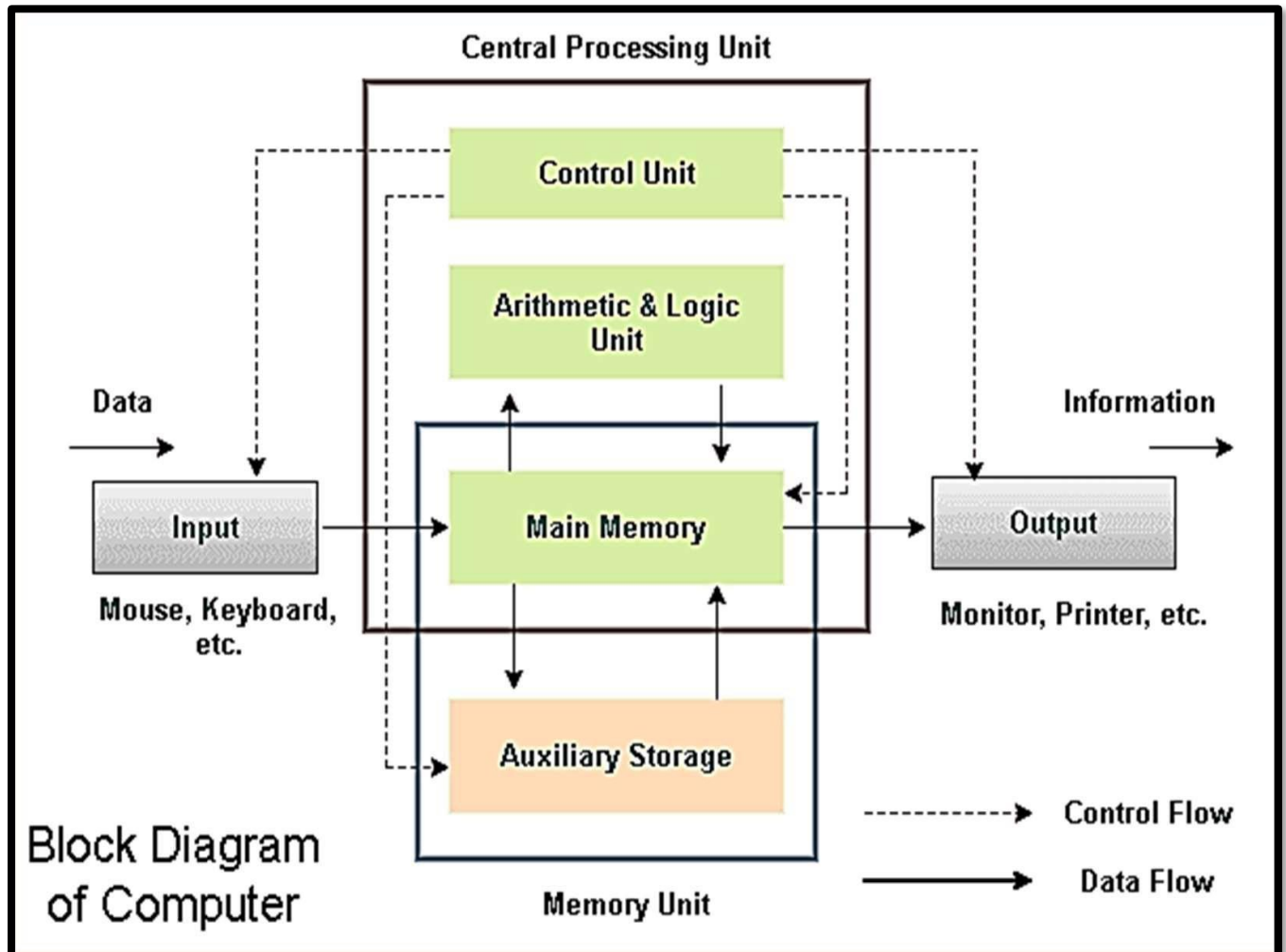


Subject Name: Programming for Problem Solving

Subject Code: BE01000121

CHAPTER NO- 1: INTRODUCTION TO PROGRAMMING

1. Basic components of a computer system



Exploring Emerging Technologies

• Central Processing Unit (CPU)

- The CPU can be called the brain of a computer system.
- The central processing unit (CPU) is where the majority of the important calculations and comparisons are performed. In addition, the CPU is in charge of turning on and controlling the operation of the other units.

- The arithmetic logic unit, also known as the ALU, and the control unit (CU) are the two primary elements that make up this unit. First, let's take a moment to briefly define these two different units of CPU.

- **Arithmetic Logic Unit (ALU)**

The arithmetic logic unit is responsible for carrying out all of the mathematical operations, including addition, subtraction, multiplication, and division. In addition to that, a logical operation is used for the comparison.

- **Control Unit (CU)**

- In addition, the control unit of a central processing unit is responsible for directing the overall operation of a computer. In addition to this, it exercises control over all devices connected to the CPU, including memory and input/output devices.
- The CU is responsible for retrieving instructions from memory, decoding those instructions, interpreting those instructions to determine what tasks are to be carried out, and then sending appropriate control signals to the other components so that they can carry out the steps necessary to execute the instruction.

- **Input/Output Unit**

Exploring Emerging Technologies

- The input/output unit is made up of different devices that are responsible for transmitting and receiving information between the memory of the computer and the outside world.
- The information that is entered into the computer via the input unit is saved in the memory of the device for later processing. The completed processing can then be saved in the memory and either recorded or displayed on the output medium.

- **Memory Unit**

- Memory units are an integral part of any modern digital computer. It is the repository for all of the results, both intermediate and final.
- The data that are read from the primary storage or an input unit are moved to the memory of the computer so that they can be processed. These data can come from either the main storage or an input unit.
- The data that needs to be processed and the instructions that need to be carried out are both stored in this memory unit so they can be accessed quickly.
- This storage unit is divided into two more sub units:

- Primary memory
- Secondary memory.

- **Primary memory:**

- This is a type of memory storage where the most recent memory is stored. In this memory only main and running activities are stored, it is smaller in size than secondary memory.
- Data in this unit is stored temporarily and the data gets wiped out once the power is cut off, so this is also called as temporary memory or main memory.
- Many of have already heard the memory type called “RAM”. So, this is also an example of primary memory.
- This memory can be directly accessed by the CPU whenever needed for the processes.
- This memory is also used for reading and writing purposes.
- If any operation is to be done on the data by the CPU, then initially that data is stored in the primary memory i.e., RAM and then processed.

- **Secondary Memory:**

- As the data stored in the primary memory is temporary, thus to store permanent data secondary data is used.
- For permanent data storage secondary memory is used. This provides the primary memory data for processing. Secondary memory is also called as auxiliary memory.
- Hard disks are the examples of the secondary memory. This memory type is less
- expensive than the primary memory.

Primary memory	Secondary memory
The primary memory of a computer is the main memory that is utilized to store data temporarily.	Secondary memory defines to additional storage devices that are utilized to store data permanently.
Primary memory is temporary.	Secondary memory is permanent.
Primary memory is faster than secondary memory because it is directly accessible to the CPU.	Secondary memory is non-volatile, which means it retains data even when the power is off.
Primary memory is directly accessible by Processor/CPU.	Secondary memory is not directly accessible by the CPU.
Nature of Parts of Primary memory varies, RAM- volatile in nature. ROM- Non-volatile.	It's always Non-volatile in nature.
Primary memory is volatile, which means it is wiped out when the computer is turned off.	Since it is non-volatile, data can be retained in case of a power failure.
Primary memory devices are more expensive than secondary storage devices.	Secondary memory devices are less expensive when compared to primary memory devices.

The memory devices used for primary memory are semiconductor memories.	The secondary memory devices are magnetic and optical memories.
It can hold data/information currently being used by the processing unit.	It can hold data/information that are not currently being used by the processing unit.
The capacity of primary memory is usually within the range of 16 to 32 GB.	It stores a considerable amount of data and information. The capacity of secondary memory ranges from 200 GB to some terabytes.
Primary memory is also known as Main memory or Internal memory.	Secondary memory is also known as External memory or Auxiliary memory.
It can be accessed by a data bus.	It can be accessed using I/O channels.
Examples: RAM, ROM, Cache memory, PROM, EPROM, Registers, etc.	Examples: Hard Disk, Floppy Disk, Magnetic Tapes, etc.

2. Hardware and Software

Hardware refers to the physical components of a computer. Computer Hardware is any part of the computer that we can touch these parts. These are the primary electronic devices used to build up the computer. Examples of hardware in a computer are the Processor, Memory Devices, Monitor, Printer, Keyboard, Mouse, and Central Processing Unit.

Types of Computer Hardware

- Input Devices
- Output Devices

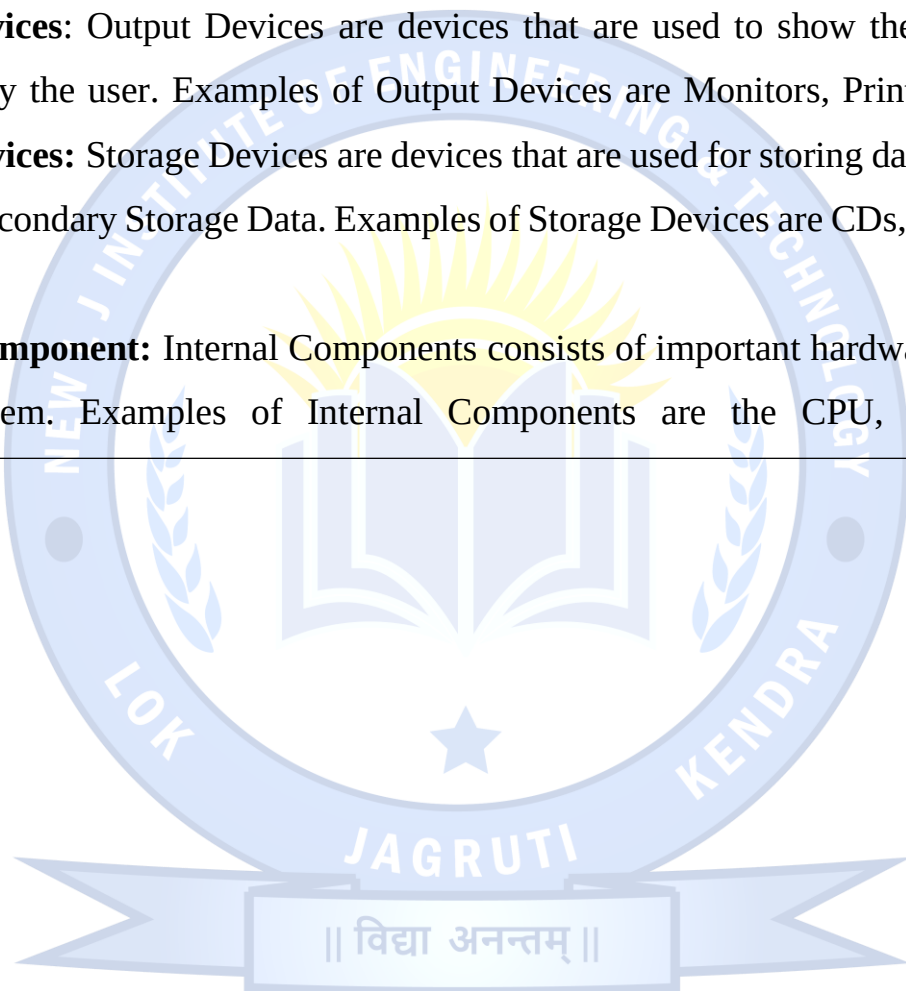
- Storage Devices
- Internal Component

Input Devices: Input Devices are those devices through which a user enters data and information into the Computer or simply, User interacts with the Computer. Examples of Input Devices are Keyboard, Mouse, Scanner, etc.

Output Devices: Output Devices are devices that are used to show the result of the task performed by the user. Examples of Output Devices are Monitors, Printers, Speakers, etc.

Storage Devices: Storage Devices are devices that are used for storing data and they are also known as Secondary Storage Data. Examples of Storage Devices are CDs, DVDs, Hard Disk, etc

Internal Component: Internal Components consists of important hardware devices present in the System. Examples of Internal Components are the CPU, Motherboard, etc.



Exploring Emerging Technologies

HARDWARE	SOFTWARE
Physical elements of a computer or electronic system	A collection of instructions that tells the computer how to perform a task
Has four main categories: input devices, output devices, secondary storage devices and internal components	Mainly divided into system software and application software
Tangible	Intangible
Developed using electronic and other materials	Developed by writing instructions using a programming language
When damaged, it can be replaced with a new component	When damaged, it can be reinstalled using a backup copy
Starts functioning once the software is loaded into the system	Should be installed into the computer to function
Ex: Keyboard, Mouse, Monitor, Printer, Hard disk, CPU, RAM, and ROM	Ex: MS Word, Excel, MSSQL, MySQL, Photoshop

Software is a collection of instructions, procedures, and documentation that performs

different tasks on a computer system. we can say also Computer Software is a programming code executed on a computer processor. The code can be machine-level code or code written for an operating system. Examples of software are MS- Word, Excel, PowerPoint, Google Chrome, Photoshop, MySQL, etc.

Types of Computer Software

- **System Software**
- **Application Software**

System Software: System Software is a component of Computer Software that directly operates with Computer Hardware which has the work to control the Computer's Internal Functioning and also takes responsibility for controlling Hardware Devices such as Printers, Storage Devices, etc. Types of System Software include Operating systems, Language processors, and Device Drivers.

Application Software: Application Software are the software that works the basic operations of the computer. It performs a specific task for users. Application Software basically includes Word Processors, Spreadsheets, etc. Types of Application software include General Purpose Software, Customized Software, etc.

Exploring Emerging Technologies

SYSTEM SOFTWARE VERSUS APPLICATION SOFTWARE

SYSTEM SOFTWARE

Computer software designed to provide a platform to other software

Manages resources and helps to run hardware and application software

Runs when the system starts and runs till the end

Developed using languages like C, C++, Assembly

Essential for the proper functioning of a system

Ex: operating system, language processors and device drivers

APPLICATION SOFTWARE

Software designed to perform a group of coordinated functions, tasks or activities for the benefit of the user

Performs a specific task according to their type

Runs when the user requires

Developed using languages like Java, C, C++, Visual Basic

Not extremely important for the functioning of the system

Ex: Word processor, Spreadsheet, Presentation software, web browsers, graphics software

3. Define following: 1) Operating System 2) Compiler 3) Interpreter 4)

Assembler 5) Loader 6) Linker 7) Pre-processor.

Operating Systems lies in the category of system software. It basically manages all the resources of the computer. An operating system acts as an interface between the software and different parts of the computer or the computer hardware. The operating system is designed in such a way that it can manage the overall resources and operations of the computer. Examples of operating systems are Windows, Linux, Mac OS, etc.

An operating system (OS) is a collection of software that manages computer hardware resources and provides common services for computer programs.

The compiler is software that converts a program written in a high-level language (Source Language) to a low-level language (Object/Target/Machine Language/0, 1's).

An interpreter is a computer program, which converts each high-level program statement into the machine code. This includes source code, pre-compiled code, and scripts.

Assembler is a program for converting instructions written in low-level assembly code into relocatable machine code and generating along information for the loader.

A linker is an important utility program that takes the object files, produced by the assembler and compiler, and other code to join them into a single executable file.

In computer systems **a loader** is the part of an operating system that is responsible for loading programs and libraries. It is one of the essential stages in the process of starting a program, as it places programs into memory and prepares them for execution.

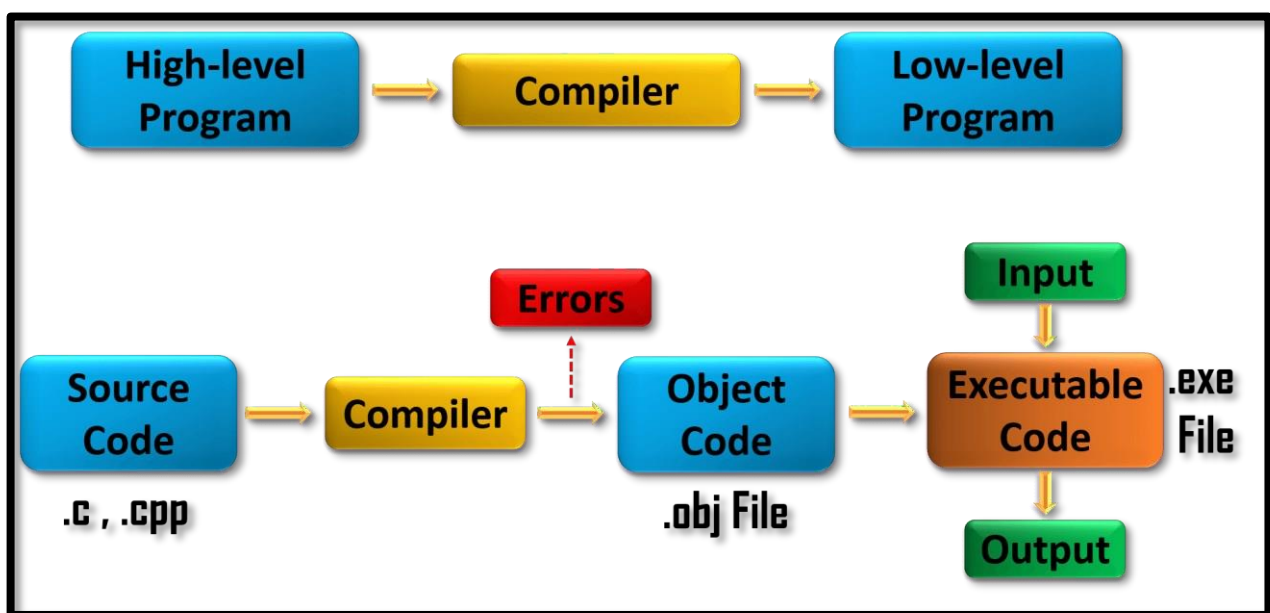
The C preprocessor is a macro preprocessor (allows you to define macros) that transforms your program before it is compiled. These transformations can be the inclusion of header files, macroexpansions, etc.

All preprocessing directives begin with a # symbol. For example, #define PI 3.14

The #include preprocessor is used to include header files to C programs. For example,
#include <stdio.h>

4. Types of Translators : Compiler, Interpreter and Assembler

Compiler



A compiler plays a crucial role in software development, as it is the tool that translates high-level programming code written by a human in a language into machine-readable instructions. This translation generally takes the form of binary code or machine code and involves several key tasks, such as lexical analysis, syntax parsing, semantic analysis, optimization, and code generation. The outcome of this process is a compiled code that can be directly executed by a computer's hardware, enabling the efficient running of software programs

A compiler works in phases: first, checking the structure of the source code to ensure it

follows the grammatical rules of the programming language. Then, it converts the code into some intermediate form while automatically bringing in some optimization for better performance. It then generates target machine code with numerous optimization techniques to enhance efficiency in executing the code. With this, the final output results in an executable program.

Here are the popular advantages of a Compiler

1. Once the code is compiled, machine code runs directly on the system, making execution faster than in interpreted languages.
2. Compilers perform optimizations during translation, leading to more efficient, faster code.
3. A compiler analyses the entire code before execution, allowing it to catch and report errors early in development.
4. The machine code generated by a compiler can be optimized for the specific hardware it will run on, improving performance.
5. Since compilers generate binary files, it's harder for unauthorized individuals to reverse-engineer or access the source code.

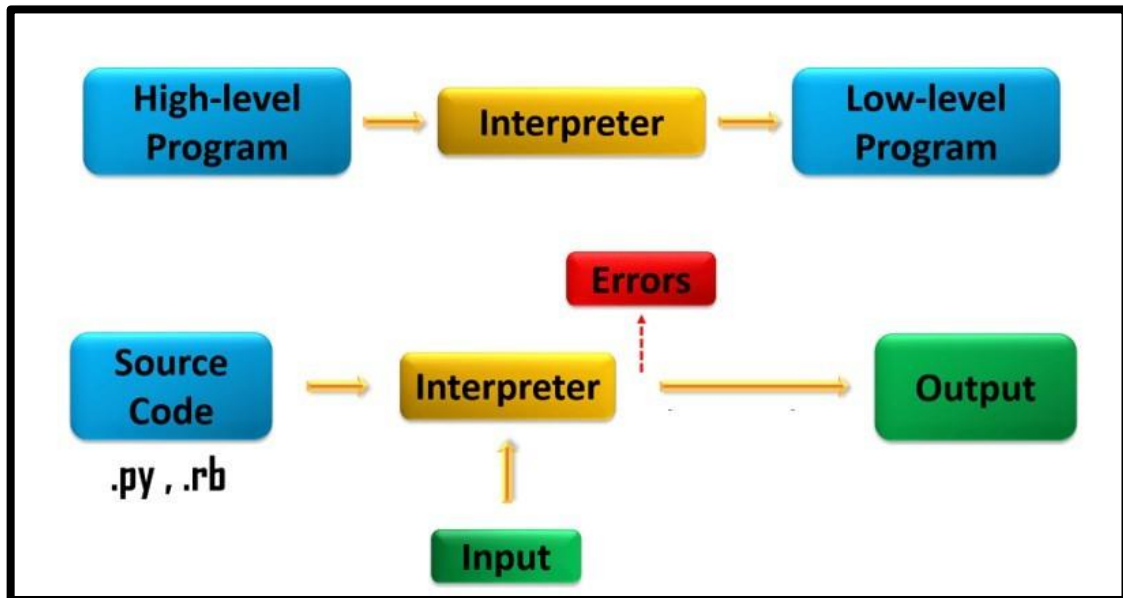
A few disadvantages of a Compiler are as follows

Compiling large programs can take time, slowing the development and testing process.

Compiled code is platform-specific, meaning the program needs to be recompiled for different hardware or operating systems.

Compilers can generate large executable files, which may consume more memory than interpreted programs.

Interpreter



An interpreter is a software tool that executes code written in a high-level programming language directly but without prior translation into machine code. It reads and executes the code line by line. It translates each line into machine instructions right then before executing the following line, making it easier to identify any errors and debug the code.

Here are the popular advantages of an Interpreter

1. Interpreters execute code line by line, allowing developers to test and debug code quickly without compiling.
2. Since interpreted code is not tied to a specific machine's architecture, it can run on multiple platforms without modification.
3. Errors are detected and displayed immediately, making it easier to identify the exact location of issues in the code.
4. Interpreters don't require the entire program to be stored in memory, as they execute

code one instruction at a time.

5. Interpreters work well with dynamically typed languages, offering more flexibility during runtime.

A few disadvantages of an Interpreter are as follows

- Interpreted programs generally run slower than compiled programs because the code is translated line by line at runtime.
- Interpreters do not optimize the code during execution, which can result in inefficient performance compared to compiled languages.
- The source code must be available on the machine where the interpreter runs, which can pose security and distribution challenges.

<u>Aspect</u>	<u>Compiler</u>	<u>Interpreter</u>
Execution Process	It translates the entire program into machine code at once.	It translates and executed code line by line.
Output	The compiler generates an executable file (machine code).	The interpreter directly executes the program without creating a separate file.
Error Detection	It detects all errors after completing the whole program.	It detects and reports errors line by line, stopping at the first error.

Speed of Execution	It is generally faster because the entire program is precompiled.	It is usually slower, as it translates code during execution.
Memory Usage	It requires more memory to store the compiled code.	It uses less memory as no executable file is generated.
Reusability	Compiled code can be executed multiple times without recompilation.	On the contrary, the Interpreter needs to re-execute the code each time.
Examples	C, <u>C++</u> , Java (JVM bytecode compiled programs)	Python, Ruby, Javascript
Use case	It is suitable for performance-critical applications.	It is ideal for scripting, development and dynamic execution.

ASSEMBLER

An assembler is a computer program that translates assembly language code into machine code, which allows a computer's hardware to be directly communicated with.

5. Types of Programming Language

- Computer languages can be categorized as low, middle, or high level, based on how

close they are to the hardware and how easy they are to use:

- **Low-level languages**
- These languages are the closest to the hardware and are also known as machine language. They are made up of only two symbols, 0 and 1, and are written in binary numbers. Low-level languages are used for system programming, device driver development, and embedded systems.

Advantages of low-level programming languages:

- **High efficiency** as programs written in low-level languages are typically more efficient regarding execution speed and memory usage.
- **Direct hardware interaction** — programmers control entirely hardware components, registers, and memory. Hence, programmers also have a high level of control over program behavior.
- **Small code size**, which can be crucial in environments with limited storage.

Disadvantages of low-level programming languages:

- **Hard learning curve** — low-level languages require a deep understanding of the hardware architecture, making learning rather challenging.
- **Time-consuming development** — writing programs in low-level languages often requires more time and effort than in high-level languages.
- **Lack of flexibility.** As mentioned, programs written in low-level languages are often specific to a particular architecture, making it difficult to transfer them to different platforms.
- **High chance of errors.** Since low-level languages provide direct access to memory and hardware, a higher risk of programming errors can lead to system crashes.

Examples of low-level programming languages:

- **Machine code.** This is the lowest level of programming language and consists of binary code that directly represents instructions executed by the CPU.
- **Assembly language.** Assembly languages are specific to a particular CPU architecture and provide a symbolic representation of machine code instructions. For example, Intel processors require x86 Assembly.
- **C:** While C was considered a high-level language a few decades ago, it now possesses low-level features like manual memory management and direct manipulation of hardware. It also lacks a sizeable runtime system and basically supports only scalar operations
- **Middle-level languages**
 - These languages are a mix of low-level and high-level languages, and are also known as intermediate-level languages. Assembly language is an example of a middle-level language.

Advantages of middle-level programming languages:

Sufficient level of abstraction. Mid-level languages provide a higher abstraction level than low-level languages so that programmers can create more intuitive and readable code.

Increased portability. Programs written in mid-level languages are more portable than those written in low-level languages. Still, they may call for some platform-specific adjustments.

Productivity. Mid-level languages are more user-friendly, which can lead to increased developers' productivity.

Versatility. Mid-level languages are suitable for a wide range of apps.

Disadvantages of middle-level programming languages:

Steep learning curve. While mid-level programming languages are generally easier to learn than low-level ones, they may still be challenging, especially for beginners.

Low control. Mid-level languages don't typically provide the same control over hardware as low-level languages do.

Dependency on libraries. Medium-level languages often rely on libraries for specific functionalities.

Examples of middle-level programming languages:

Dialects of C that allow raw pointers. Technically, C++ can be considered as medium-level programming language as well.

The Wirth family of languages. Such languages as Pascal, Modula2, and Oberon.

High-level languages

These languages are more abstract from the hardware and closer to human language. They are used for software development, web development, and database management. Examples of high-level languages include Python, C++, C, C#, Visual Basic, and JavaScript.

Advantages of high-level programming languages:

Ease of learning and use. High-level languages often feature English-like syntax, making them easier for beginners to learn and use.

Enhanced productivity. No wonder programming in high-level languages is faster and more efficient than low-level/mid-level languages.

Extra portability. Programs written in high-level languages are often portable across different platforms with little or no modification.

Quick development. High-level languages often include features like dynamic typing, automatic memory management (garbage collection), and built-in libraries to facilitate the development process for programmers.

No complex operations. High-level languages lack tricky operations, such as memory management and hardware-specific details. This allows developers to focus on solving problems at a higher level.

Disadvantages of high-level programming languages:

Less control over hardware. High-level programming languages abstract many hardware details and give programmers less control over the computer's resources.

High resource consumption. Applications written in high-level languages may consume many system resources regarding memory and processing power.

Dependency on libraries and frameworks. High-level languages often rely on libraries and frameworks for various functionalities.

Examples of high-level programming languages in active use today include JavaScript, Python, Java, Visual Basic, Perl, Ruby, C#, and many others.



Exploring Emerging Technologies